Debugging & Optimisation of Computer Arithmetic

Course: Computer Architecture (CSA12) | CO2 — Integer & Floating-Point Arithmetic Algorithms | Assessment Tool 2

Each task below follows the same flow: the **bug or bottleneck is identified**, the fault is traced through a **bit-level worked example**, a **corrected or optimised solution** is given, and the result is **verified**. Notation used in the code blocks: (+) = XOR, . = AND, ASR = arithmetic shift right, Shl = shift left. All numbers are 2's-complement unless stated.

Bug identified: +120 and +50 both fit in 8 bits individually, but their true sum (+170) exceeds the signed 8-bit range [−128, +127]. The hardware still produces a correct bit pattern, but interpreted as signed it reads −86. This is a **signed-overflow** condition that the program is not detecting.

Root-Cause Analysis

+120 = 0111 1000 and +50 = 0011 0010. Adding them bit-by-bit:

```
0 1 1 1 1 0 0 0 (+120)
+ 0 0 1 1 0 0 1 0 (+50)
-------------------
1 0 1 0 1 0 1 0 = 170 (unsigned) = -86 (signed)

Carry INTO the MSB (bit 7) = 1
Carry OUT of the MSB (bit 7) = 0
Overflow V = Cin (+) Cout = 1 (+) 0 = 1 -> OVERFLOW
```

Two independent tests both confirm the overflow:

Sign rule: two like-signed operands produce an unlike-signed result	(+) plus (+) gives a result whose sign bit = 1 (negative)	Overflow
Carry rule: V = Cin XOR Cout at the MSB	1 XOR 0 = 1	Overflow

Note the distinction between the two flags: the **carry flag C = 0** (170 < 256, so there is no unsigned overflow), while the **overflow flag V = 1** (signed overflow). Testing the wrong flag — e.g. relying on C for a signed operation — is the underlying programming error.

Corrected / Optimised Solution

- **Detect** signed overflow with V = C_in(MSB) XOR C_out(MSB), or equivalently flag it whenever both operands share a sign that differs from the result's sign.
- **Handle** it by widening the operands before adding — sign-extend to 16 bits so the sum fits:

```
0000 0000 0111 1000 (+120, sign-extended)
+ 0000 0000 0011 0010 (+50, sign-extended)
---------------------------
0000 0000 1010 1010 = +170 (MSB = 0 -> positive, CORRECT)
```

Alternatively, in fixed-width hardware, raise the V flag and either trap the exception or saturate the result to +127 so the error is contained rather than silently wrong.

Verification: the 8-bit add reads −86 (wrong) and the CPU asserts V = 1; after 16-bit sign extension the sum is +170 with V = 0. Overflow is now both detected and handled correctly.

Bottleneck identified: in a ripple-carry adder each stage's carry-out feeds the next stage's carry-in, so the carry must propagate serially from the LSB to the MSB. The MSB sum cannot settle until the carry has rippled through every preceding stage, making the worst-case delay grow linearly with the word length.

Root-Cause Analysis — how the carry propagates

```
Full adder i:
Si = Ai (+) Bi (+) Ci ( (+) = XOR )
Ci+1 = Gi + Pi . Ci
where Gi = Ai . Bi <- "generate" (makes a carry on its own)
Pi = Ai (+) Bi <- "propagate" (passes an incoming carry)

Dependency chain (ripple):
C1 -> C2 -> C3 -> ... -> Cn (each Ci needs the previous one)
Worst-case delay ~ 2n gate delays => O(n)
```

For a 32- or 64-bit adder in a real-time loop this linear critical path dominates the clock period and throttles every arithmetic instruction.

Optimised Solution — Carry-Look-Ahead Adder (CLA)

Expand the recurrence Ci+1 = Gi + Pi.Ci so that every carry is a direct two-level (AND–OR) function of the inputs and C0 — no waiting on the previous carry:

```
C1 = G0 + P0.C0
C2 = G1 + P1.G0 + P1.P0.C0
C3 = G2 + P2.G1 + P2.P1.G0 + P2.P1.P0.C0
C4 = G3 + P3.G2 + P3.P2.G1 + P3.P2.P1.G0 + P3.P2.P1.P0.C0

All Gi, Pi ready 1 gate delay after inputs.
All carries then computed IN PARALLEL in 2 more gate delays.
Sums Si = Pi (+) Ci follow 1 delay later.
```

Because the carries are generated simultaneously instead of sequentially, the ripple is eliminated:

Ripple-carry	~ 2n gate delays (O(n))	serially, LSB to MSB
Carry-look-ahead (4-bit block)	~ 3-4 gate delays (O(1))	all carries in parallel from G, P, C0
Hierarchical / block CLA (large n)	O(log n)	4-bit CLA blocks combined in a tree

Why it is faster: ripple delay scales with n, whereas a CLA block resolves all its carries in a fixed number of gate levels. The trade-off is more gates and higher fan-in, so real designs cascade small 4-bit CLA blocks (block-carry-look-ahead) to keep fan-in bounded while achieving an O(log n) overall delay.

Verification: for a 16-bit add, ripple needs about 32 gate delays on the critical path; a 4-level block CLA settles the same carries in about 8–10 — a 3–4× speed-up for identical sums, which removes the real-time bottleneck.

Bug identified: Booth's algorithm relies on an **arithmetic shift right** (the sign bit is replicated) so that partial products keep their sign. If the shift is coded as a logical shift (zero-fill), the sign information is destroyed as soon as a partial product goes negative — exactly when a negative operand is involved. Related faults are swapping the add/subtract cases and failing to initialise $Q_{-1} = 0$.

The Booth Recoding Rules

0	0	No add / subtract -> shift only
1	1	No add / subtract -> shift only
0	1	A = A + M (end of a run of 1s), then ASR
1	0	A = A - M (start of a run of 1s), then ASR

Correct Step-by-Step Execution

Worked example (4-bit): M = −3 = 1101, multiplier Q = +2 = 0010, −M = 0011, n = 4 iterations. Registers A, Q are 4 bits; Q-1 is the appended bit.

Init	--	Initialise	0000	0010	0
1	0 0	ASR only	0000	0001	0
2	1 0	A=A-M; ASR	0001	1000	1
3	0 1	A=A+M; ASR	1111	0100	0
4	0 0	ASR only	1111	1010	0

Product = A:Q = 1111 1010 = −6 (decimal). Check: −3 × +2 = −6. **Correct.**

Where the Bug Bites

In steps 2 and 3 the accumulator holds negative values. A correct **ASR** replicates the sign bit; a **logical** shift would instead inject 0s at the top:

```
Step 3 (0 1): after A = A + M -> A = 1110 (should ASR, sign=1)
correct ASR -> A = 1111 (sign preserved)
logical shift -> A = 0111 (sign LOST, looks positive!)

Final product with the logical-shift bug:
A:Q = 0011 1010 = +58 <- WRONG (positive)
Correct product:
A:Q = 1111 1010 = -6
```

Corrections

- Use an **arithmetic** shift right on the combined [A : Q : Q-1] register so the sign bit of A is replicated every step.
- Keep A sign-extended when performing A ± M (the multiplicand M and its negation must also be sign-extended to A's width).
- Initialise $Q_{-1} = 0$ and run exactly n iterations for n-bit operands.
- Map the cases correctly: 01 gives add, 10 gives subtract (swapping them silently corrupts results).

Verification: with the arithmetic shift restored, −3 × +2 yields 1111 1010 = −6, and the same program now returns correct products for both operand signs (e.g. −3 × −2 = +6). The buggy logical shift had returned +58.

Inefficiency identified: restoring division subtracts the divisor from the partial remainder and, whenever the result turns negative, adds the divisor back (the 'restore' step) before moving on. That extra addition can occur in every iteration, so an n-bit division may perform up to **2n** add/subtract operations.

Root-Cause Analysis

Restoring loop per bit: shift, subtract divisor, test sign; if negative, restore (add divisor) and record quotient bit 0, else record 1. The restore is pure overhead — work done only to undo the trial subtraction — and it lengthens the critical path of an embedded divider.

Optimised Solution — Non-Restoring Division

Never restore. Instead, remember the sign and compensate on the next step: subtract when the remainder is non-negative, add when it is negative. Only one add/subtract per iteration.

```
Init: A = 0, Q = dividend, M = divisor, count = n

Repeat n times:
if A >= 0: Shl(A,Q); A = A - M
else : Shl(A,Q); A = A + M
if A >= 0: Q0 = 1 else Q0 = 0

Final correction:
if A < 0: A = A + M (one restore, at the very end)

Result: Q = quotient, A = remainder
```

Worked Example & Verification

Divide 7 by 3 (4-bit): M = 0011, −M = 1101, dividend Q = 0111, A = 0000, n = 4. The sign tested is A's sign at the start of each iteration.

Init	--	A=0, Q=0111	0000	0111
1	>= 0	Shl; A=A-M; A<0 so Q0=0	1101	1110
2	< 0	Shl; A=A+M; A<0 so Q0=0	1110	1100
3	< 0	Shl; A=A+M; A>=0 so Q0=1	0000	1001
4	>= 0	Shl; A=A-M; A<0 so Q0=0	1110	0010
Corr	< 0	A=A+M (final restore)	0001	0010

Quotient Q = 0010 = 2, Remainder A = 0001 = 1. Check: 7 = 3 × 2 + 1. **Correct.**

Performance Comparison

Add/subtract per iteration	1 subtract + possible 1 restore	exactly 1 (add or subtract)	

Worst-case operations (n bits)	up to 2n	n + 1 (one final correction)	
This example (7 / 3)	4 subtracts + 3 restores = 7	4 + 1 correction = 5	
Effect	extra additions lengthen each cycle	fewer ops, faster, shorter path	

Verification: both methods return quotient 2, remainder 1, but non-restoring reaches it in 5 arithmetic operations versus 7 — roughly halving the additions in the worst case and removing the wasted restore work each cycle.

Bug identified: to add two floats their exponents must be aligned by shifting the smaller number's mantissa right. Single precision keeps only 24 bits of significand (1 hidden + 23 stored). When the exponent difference exceeds about 24, every significant bit of the small operand is shifted out — it is **swamped** and contributes nothing. This is loss of significance, not a coding typo.

IEEE 754 Single-Precision Layout

Sign	1	bit 31
Exponent	8	biased by 127
Mantissa (fraction)	23	+ 1 hidden leading bit = 24-bit precision

Machine epsilon (single) is 2^−23, about 1.19×10^−7. Any addend smaller than roughly epsilon times the larger value cannot change it.

Root-Cause Analysis — worked example

16,777,216 = 2^24 in single precision:
0 10010111 00000000000000000000000
sign=0 exp=151 (=127+24) mantissa=0 -> 1.0 x 2^24

Add 1.0 (= 1.0 x 2^0):
exponent difference = 24
-> shift the 1.0 mantissa right by 24 bits to align
-> its leading 1 falls OUTSIDE the 24-bit significand

Spacing (ULP) at 2^24 = 2^(24-23) = 2
representable neighbours: ... 2^24, 2^24 + 2, 2^24 + 4 ...
2^24 + 1 sits exactly halfway -> round-to-nearest-even -> 2^24

RESULT: 16,777,216 + 1 = 16,777,216 (the +1 is LOST)

The same effect explains why summing many tiny values into a large accumulator loses them one by one: alignment discards their bits, normalisation finds nothing new, and round-to-nearest leaves the large value unchanged.

Optimisation Techniques

• **Use higher precision:** double precision has a 52+1 = 53-bit significand (epsilon about 2.22×10^−16), tolerating exponent gaps up to about 53 before swamping; extended/quad precision extends this further. • **Kahan (compensated) summation:** carry a running correction term that recovers the low-order bits lost in each add, cutting the error growth from roughly n×epsilon to about epsilon.

• **Pairwise or sorted (smallest-first) summation:** combine numbers of similar magnitude before adding them to much larger ones, so small values are not swamped prematurely.

• **Guard, round and sticky bits** in the FP adder preserve information about the shifted-out portion for correct rounding, and **rescaling** can factor out a common magnitude.

Kahan summation (recovers lost low-order bits):
sum = 0 ; c = 0 // c = compensation
for each x:
y = x - c // add back last correction
t = sum + y
c = (t - sum) - y // what was actually lost
sum = t

Verification: in single precision 2^24 + 1 returns 2^24 (the addend vanishes). Doing the same in double precision returns 16,777,217 exactly; and Kahan summation lets the small terms accumulate correctly even in single precision — both restore accuracy.